

Multithreading

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor (CSE), PIT
Parul University

1.	Introduction to Threads.....	1
2.	Thread Priority and Scheduling	3
3.	Summary of Key Concepts.....	4
4.	References.....	5

9.1.1 Introduction to Threads

✓ What is a Thread?

- A **thread** is a lightweight, smallest unit of a process that executes independently.
- It allows **concurrent execution** of two or more parts of a program for maximum utilization of CPU.
- Java supports **multithreading**, which enables multiple threads to run concurrently within a single program.

✓ Difference Between Process and Thread

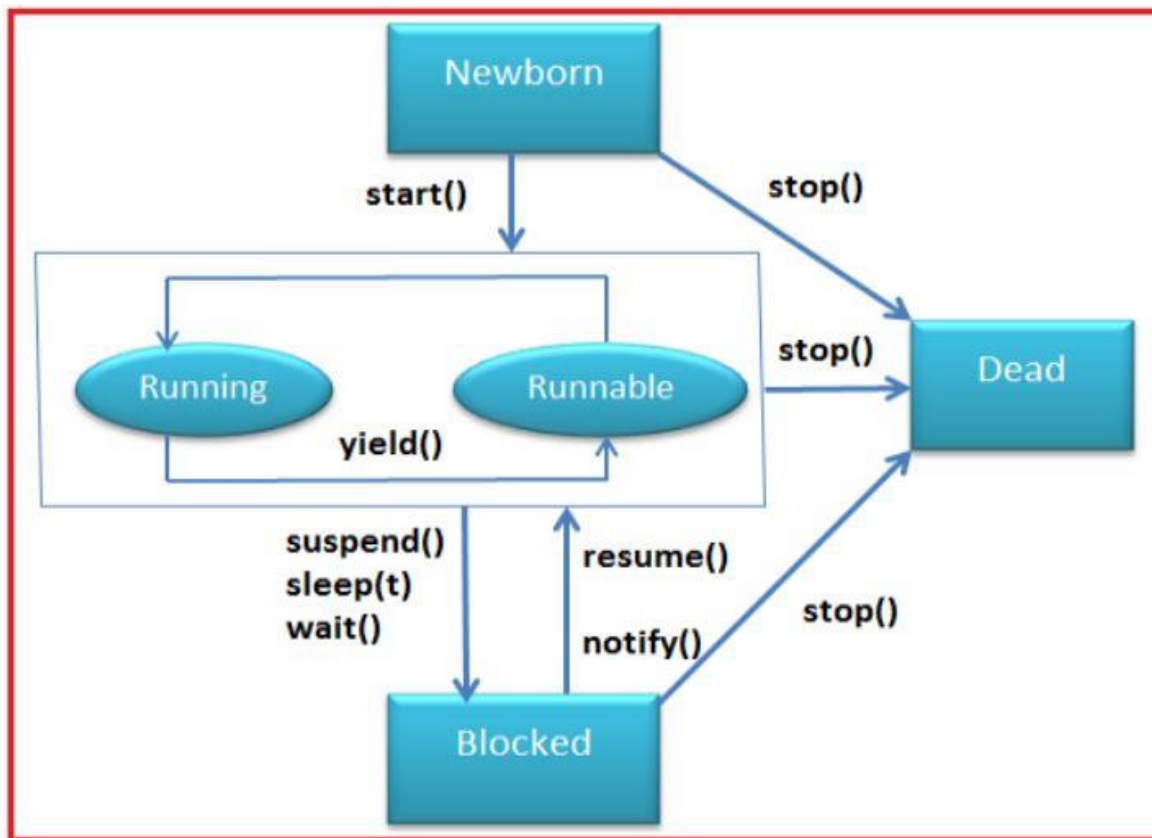
Feature	Process	Thread
Definition	A program in execution	A unit of execution within a process
Memory	Separate memory for each process	Shares memory with other threads
Communication	Inter-process communication is complex	Easier due to shared memory
Overhead	Heavyweight	Lightweight
Creation	More time and resources	Less time and resources

✓ Thread Lifecycle in Java

A thread in Java goes through the following states:

1. **New**: When an instance of the thread is created.
2. **Runnable**: After start() is called; eligible to run.
3. **Running**: Actively executing.
4. **Blocked/Waiting**: Waiting for a resource or condition.
5. **Terminated**: Completed or aborted execution.

 Diagram:



Source: <https://dotnettutorials.net/wp-content/uploads/2020/08/Thread-Life-Cycle-in-Java.jpg>

 Creating Threads in Java

 Method 1: By Extending Thread Class

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread running...");  
    }  
  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start(); // Calls run() method  
    }  
}
```

◆ Method 2: By Implementing Runnable Interface

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Runnable thread running...");  
    }  
  
    public static void main(String[] args) {  
        Thread t = new Thread(new MyRunnable());  
        t.start();  
    }  
}
```

Key Notes:

- Use **Thread class** when you don't need to extend another class.
- Prefer **Runnable interface** for flexibility (Java supports only single inheritance).
- Calling `start()` starts a new thread, which calls the `run()` method internally.
- Threads improve performance in I/O-heavy or concurrent tasks.

9.1.2 Thread Priority and Scheduling

- **Thread priority** determines the order in which threads are scheduled for execution.
- Range: `MIN_PRIORITY` (1) to `MAX_PRIORITY` (10), default is `NORM_PRIORITY` (5).
- Set using `.setPriority(int)`

```
Thread t = new Thread();  
t.setPriority(Thread.MAX_PRIORITY); // Set to 10
```

- **Thread Scheduler** is part of the JVM, which decides which thread to run.
 - In most JVMs, it's **preemptive** and **priority-based**.

Note: Thread priority doesn't guarantee execution order; it's a hint to the JVM.

✓ Daemon vs. User Threads

Feature	User Thread	Daemon Thread
Purpose	Core tasks (e.g., main thread)	Background tasks (e.g., garbage collector)
JVM waits for them?	Yes, JVM waits for all to finish	No, JVM exits when only daemon threads remain
Created with	Default thread type	<code>setDaemon(true)</code> before <code>.start()</code>

Example:

```
Thread daemonThread = new Thread(() -> {  
    while(true) {  
        System.out.println("Daemon thread running...");  
    }  
});  
daemonThread.setDaemon(true); // Must be called before start()  
daemonThread.start();
```

➤ Summary of Key Concepts:

- A thread is a smaller, independent part of a process.
- Java allows creating threads using Thread class or Runnable interface.
- Threads share memory and offer lightweight, concurrent execution.
- Thread lifecycle includes New, Runnable, Running, Waiting, and Terminated states.
- Threads are started using `start()`, and they execute `run()` method.
- Java threads go through multiple states from New to Terminated.
- Thread priority helps suggest execution order to the JVM.
- User threads perform primary tasks; daemon threads run background tasks.
- Daemon threads are terminated automatically when user threads finish.

➤ References:

- ❑ Oracle Java Documentation ❑ Threads and Concurrency
🔗 <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- ❑ GeeksforGeeks ❑ Multithreading in Java
🔗 <https://www.geeksforgeeks.org/multithreading-in-java/>
- ❑ Baeldung ❑ Java Threads
🔗 <https://www.baeldung.com/java-threads>
- ❑ JavaTpoint ❑ Java Thread Lifecycle
🔗 <https://www.javatpoint.com/lifecycle-of-a-thread>
- ❑ Oracle Java Documentation ❑ Thread Control
🔗 <https://docs.oracle.com/javase/tutorial/essential/concurrency/index.html>
- ❑ GeeksforGeeks ❑ Java Thread States
🔗 <https://www.geeksforgeeks.org/java-thread-life-cycle/>
- ❑ Baeldung ❑ Thread Priorities and Daemon Threads
🔗 <https://www.baeldung.com/java-thread-priority>
🔗 <https://www.baeldung.com/java-daemon-thread>
- ❑ JavaTpoint ❑ Java Thread Priority and Daemon
🔗 <https://www.javatpoint.com/daemon-thread>

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | **GRADE**



    
<https://paruluniversity.ac.in/>